Predicting Diabetes from Health Indicators

An End-to-End Predictive Analytics Pipeline
Using the CDC BRFSS 2015 Dataset

Code Repository: Diabetes Prediction Report

MSIN0097 — Predictive Analytics
Candidate Number: VXYY7

Word Count: ~ 2,000

1. Problem Framing & Dataset

This project focuses on a binary classification task: predicting whether an individual has diabetes or prediabetes using routinely collected health survey data. The dataset comes from the CDC's Behavioral Risk Factor Surveillance System (BRFSS) 2015 annual survey and includes responses from 253,680 individuals. It contains 21 predictor variables covering demographics, lifestyle behaviors, and self-reported health conditions (CDC, 2015).

The target variable, Diabetes_binary, is coded as 1 for respondents who reported being diagnosed with diabetes or prediabetes and 0 otherwise. Diabetes affects roughly 14% of adults in the United States and is associated with substantial healthcare costs (American Diabetes Association, 2018), making early risk identification an important and valuable use case for predictive analytics.

Success Metrics & Constraints

The primary evaluation metric used in this project is ROC-AUC, as it is well suited for datasets with class imbalance. The F1-score is used as a secondary metric because it provides a balance between precision and recall. In addition, recall for the positive class is monitored separately since, in a healthcare screening context, false negatives—failing to identify someone who may have diabetes—are generally more costly than false positives.

Several practical constraints are also considered. The model should remain interpretable so that healthcare stakeholders can understand and trust the results. Fairness across demographic groups, particularly sex and income, is also assessed to ensure the model does not disproportionately disadvantage certain populations. Finally, the analysis is designed to be fully reproducible, using fixed random seeds and a defined environment file.

Assumptions & Limitations

Key assumptions in this analysis include that the self-reported survey responses are generally accurate, that the predictor variables reflect existing health indicators rather than outcomes of a diabetes diagnosis, and that the cross-sectional nature of the survey data supports identifying associations rather than causal relationships.

One important limitation is that the target variable groups diabetes and prediabetes into a single category. While this simplifies the prediction task, it may blur important differences between the two conditions and potentially weaken the predictive signal for each.

2. Exploratory Data Analysis

2.1 Class Imbalance

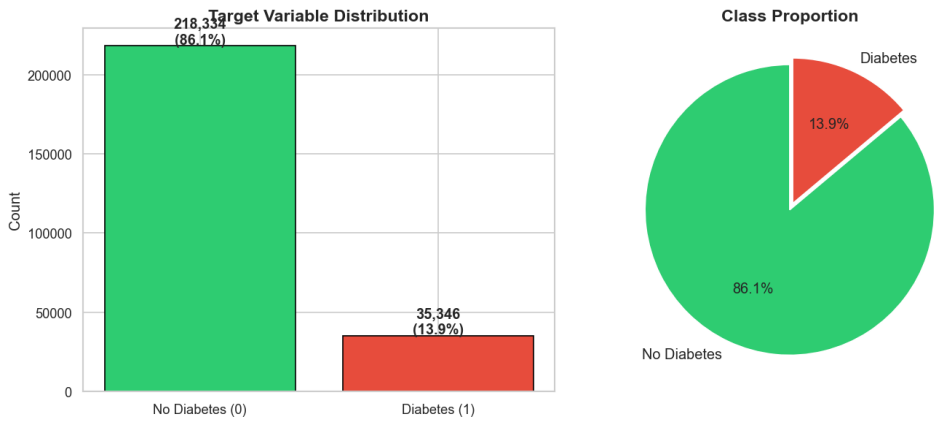


Figure 1: Target variable distribution showing 86.1% non-diabetic vs 13.9% diabetic respondents.

The dataset shows a clear class imbalance. Of the 253,680 respondents, 218,334 (86.1%) are non-diabetic, while 35,346 (13.9%) report diabetes or prediabetes, resulting in an imbalance ratio of roughly 6.2:1 (Figure 1). Because of this, a simple classifier that always predicts the majority class would already achieve 86.1% accuracy, which highlights why accuracy alone is not a useful evaluation metric for this problem.

The distribution is broadly consistent with CDC estimates of diabetes prevalence in the U.S. (~14%), suggesting the sample is reasonably representative. Given this imbalance, the modelling approach incorporates SMOTE oversampling on the training data and focuses on more appropriate evaluation metrics such as ROC-AUC and F1-score.

2.2 Feature Distributions by Diabetes Status

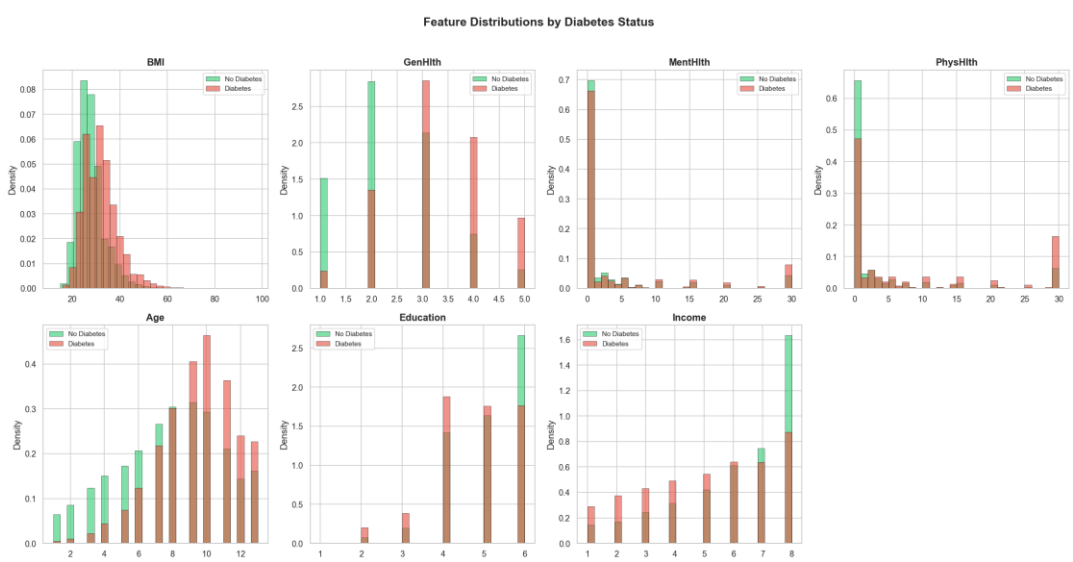


Figure 2: Density plots of continuous and ordinal features stratified by diabetes status.

Figure 2 shows noticeable differences in the distributions of several features between the diabetic and non-diabetic groups. BMI, for example, is clearly shifted to the right for individuals with diabetes, with a mode around 30–35 compared to 25–28 for non-diabetics. This aligns with the well-established relationship between obesity and diabetes. GenHlth (self-reported general health, where 1 = excellent and 5 = poor) displays the strongest visual separation: most non-diabetic respondents fall between 1 and 3, while the diabetic group shifts noticeably toward 4 and 5. While this pattern is clinically plausible, it also raises the possibility that GenHlth may partly capture respondents' awareness of their diagnosis, rather than acting as a fully independent predictor.

Age shows a similarly clear pattern. Older categories (9–12, roughly ages 55–75+) are much more common among respondents with diabetes. MentHlth and PhysHlth are both heavily zero-inflated, meaning many respondents report zero days of poor health, but the diabetic group has a noticeably longer tail, with more individuals reporting 15–30 days of poor mental or physical health. Finally, income reveals a socioeconomic gradient: non-diabetic respondents are more concentrated in the highest income brackets (7–8), whereas individuals with diabetes are more evenly distributed and more frequently represented in lower income categories (1–4).

2.3 Binary Feature Prevalence

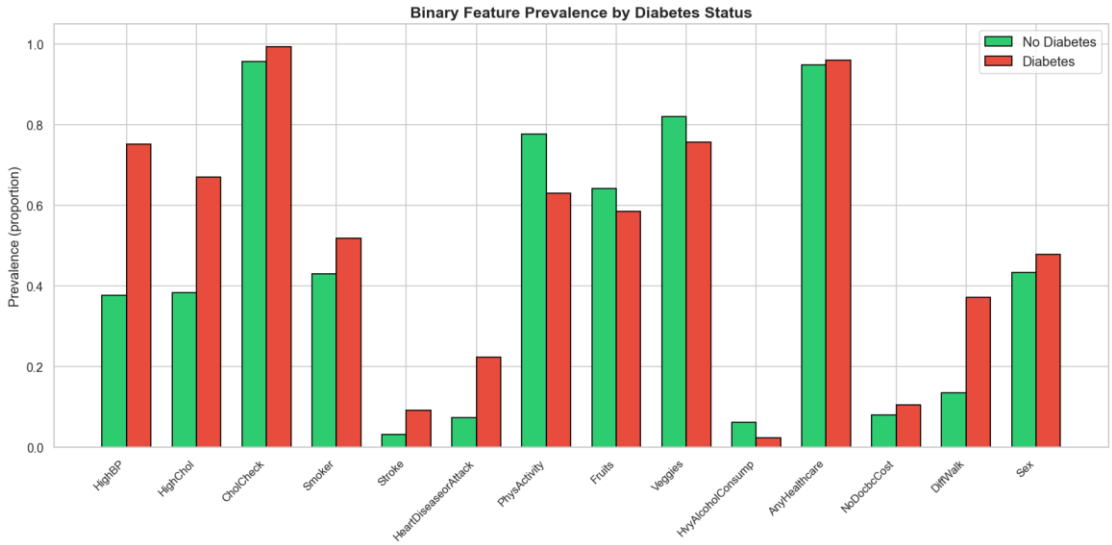


Figure 3: Prevalence of binary health indicators stratified by diabetes status.

Figure 3 highlights the binary features that differ most between the diabetic and non-diabetic groups. HighBP shows the largest gap, with a prevalence of about 75% among diabetics compared to 38% among non-diabetics—nearly double. HighChol follows a similar pattern (67% vs 39%), reinforcing the idea that conditions associated with metabolic syndrome often occur together. DiffWalk also shows a substantial difference, with a roughly threefold increase among diabetics (37% vs 13%). Likewise, HeartDiseaseorAttack rises from 8% in non-diabetics to 23% in the diabetic group, consistent with diabetes being a major cardiovascular risk factor.

In contrast, PhysActivity is noticeably lower among respondents with diabetes (63% vs 78%), suggesting it may act as a protective factor. CholCheck and AnyHealthcare are both close to ceiling levels for both

groups (above 95%), indicating they offer little discriminative value for the model. Finally, HvyAlcoholConsump is slightly lower among diabetics (3% vs 6%), which may reflect lifestyle changes following diagnosis rather than a genuinely protective effect.

2.4 Correlation Structure & Leakage Assessment

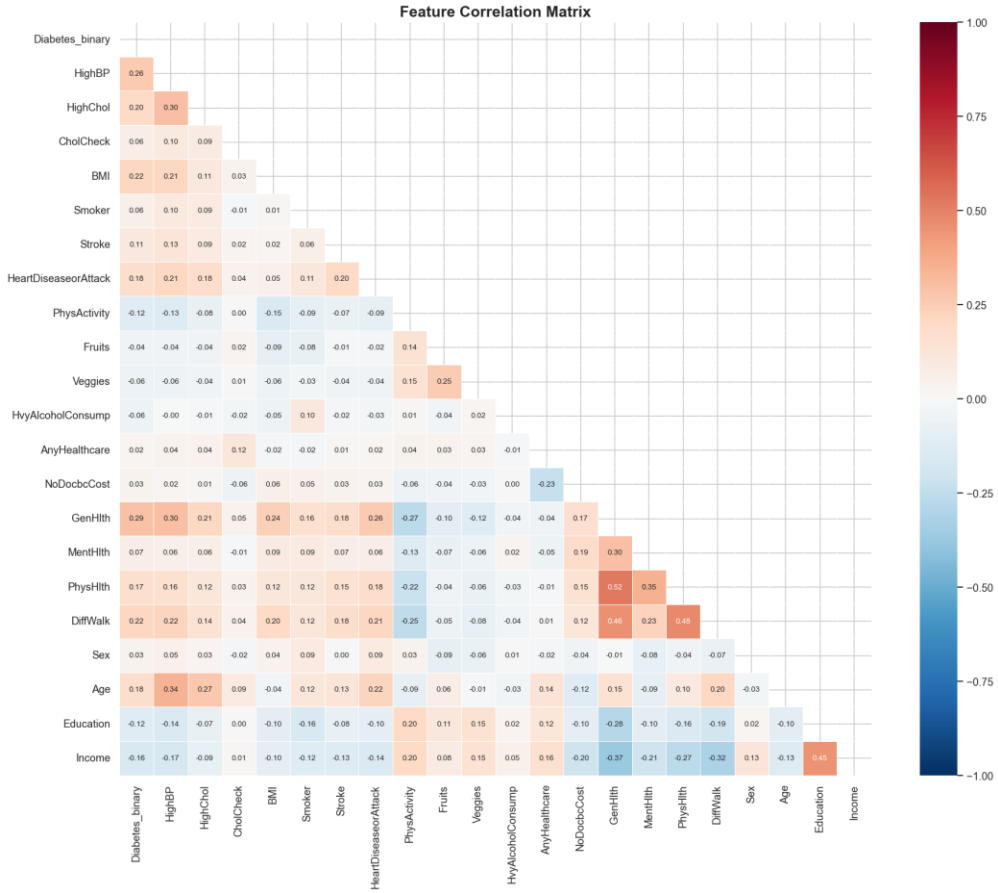


Figure 4: Pearson correlation heatmap for all features.

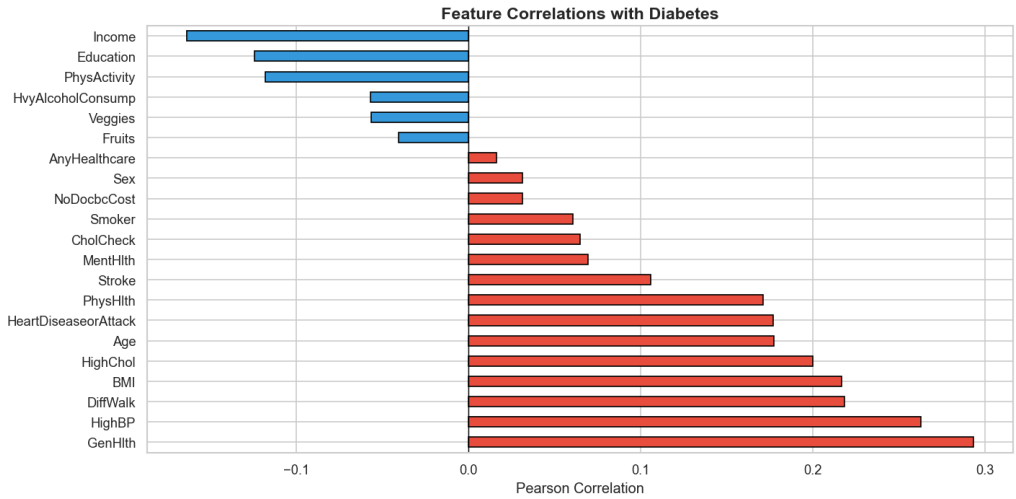


Figure 5: Feature correlations with the target variable, ranked by magnitude.

The correlation matrix (Figure 4) shows that no pairwise correlations exceed 0.52, suggesting that severe multicollinearity is not present in the dataset. The strongest relationships between features appear between PhysHlth and MentHlth (0.52), PhysHlth and DiffWalk (0.48), and Education and Income (0.45). GenHlth emerges as a central variable in the network of relationships, showing positive correlations with risk-related features such as HighBP (0.30) and DiffWalk (0.46), while correlating negatively with factors generally associated with better health outcomes, including Income (−0.37) and PhysActivity (−0.27).

When looking specifically at correlations with the target variable (Figure 5), GenHlth has the strongest association (r = 0.29), followed by HighBP (0.26), DiffWalk (0.22), and BMI (0.22). Importantly, no feature has a correlation greater than |0.5| with the target, which suggests a low risk of data leakage. Overall, the relatively modest correlations between individual features and the outcome support the use of non-linear ensemble models, which are better suited to capturing interaction effects that simple Pearson correlations might overlook.

2.5 Outlier Analysis

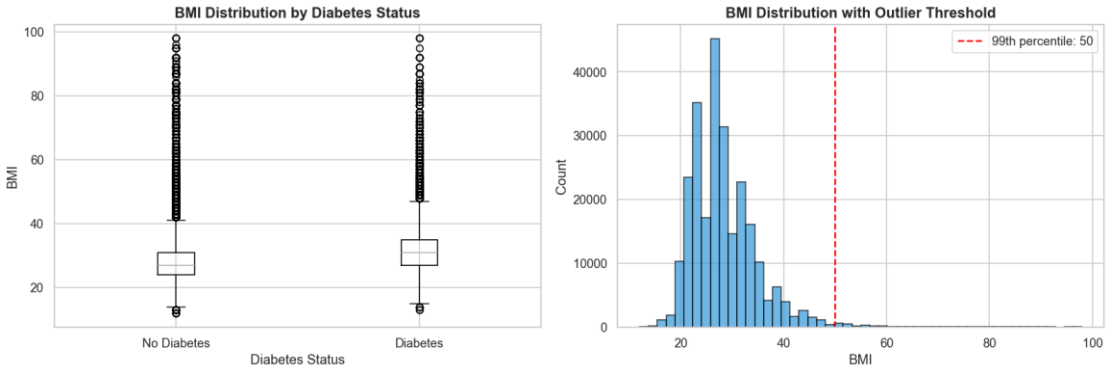


Figure 6: BMI distribution by diabetes status (left) and outlier threshold at the 99th percentile (right).

Figure 6 shows that the diabetic group has a higher median BMI of around 31, which falls in the obese range, compared to about 27 for the non-diabetic group, which is considered overweight. Both groups also contain extreme outliers, with BMI values reaching as high as 95–98. In total, 805 observations (0.32%) exceed a BMI of 60.

To manage these extreme values, the 99th percentile (BMI = 50) was used as a Winsorisation cap. Rather than removing these observations, capping was chosen so that individuals with very high BMI, who may also be at higher risk, remain in the dataset. Importantly, the threshold was calculated using training data only, helping to avoid any potential data leakage into the test set.

3. Data Preparation

Data preparation followed a split-first approach to prevent data leakage. The dataset was divided into training (60%, 152,208 rows), validation (20%, 50,736), and test (20%, 50,736) sets using stratified sampling, ensuring the original 13.9% positive class ratio was preserved across all splits. Four automated validation checks were then run to confirm that no values were missing, indices did not overlap between splits, class stratification was maintained, and all feature values fell within valid ranges.

BMI values were capped at the 99th percentile (49.0) based on the training data. A StandardScaler was also fit only on the training set and then applied to the validation and test sets. To address class imbalance, SMOTE was applied exclusively to the training data, balancing both classes to 131,000 observations each. This pipeline ensured that neither scaling parameters nor synthetic samples introduced information from the validation or test data, preserving the integrity of the evaluation.

4. Model Exploration & Shortlisting

Five models were trained on the SMOTE-augmented training set and evaluated on the validation set:

Model	Accuracy	F1-Score	ROC-AUC	Avg Precision
Logistic Regression	0.7342	0.4425	0.8218	0.4142
Random Forest	0.8179	0.4599	0.8208	0.4125
XGBoost	0.8664	0.2882	0.8259	0.4298
LightGBM	0.8659	0.2849	0.8258	0.4288
Neural Network	0.7569	0.4432	0.8125	0.4106

Table 1: Model comparison on the validation set.

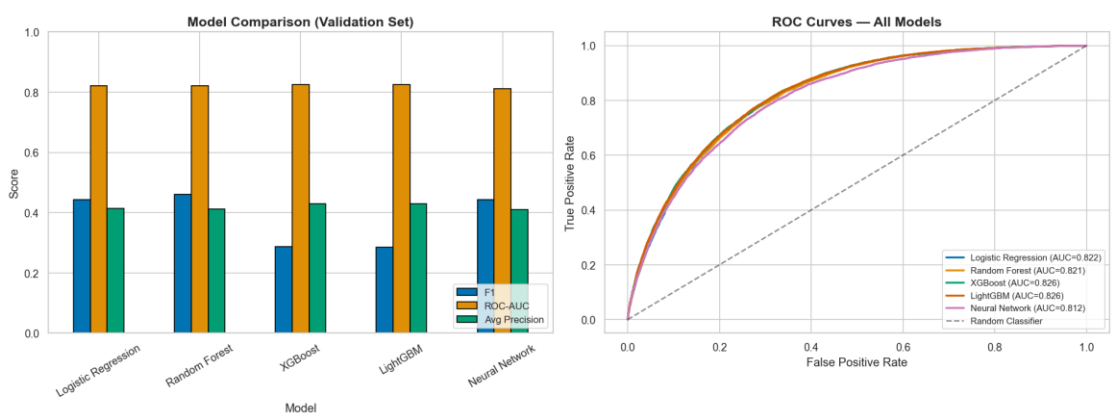


Figure 7: Model comparison bar chart and ROC curves on the validation set.

Table 1 and Figure 7 highlight an important trade-off in model performance. Gradient boosting models achieved the highest ROC-AUC (around 0.826) and accuracy (about 0.866), but their F1-scores were

noticeably lower (around 0.29) compared with Logistic Regression (0.44) and Random Forest (0.46). This pattern likely reflects a threshold effect, where the boosting models learned more conservative decision boundaries that favour specificity over sensitivity.

The neural network showed a different profile: it achieved a competitive F1-score (0.44) but produced the lowest ROC-AUC (0.813) among the models tested.

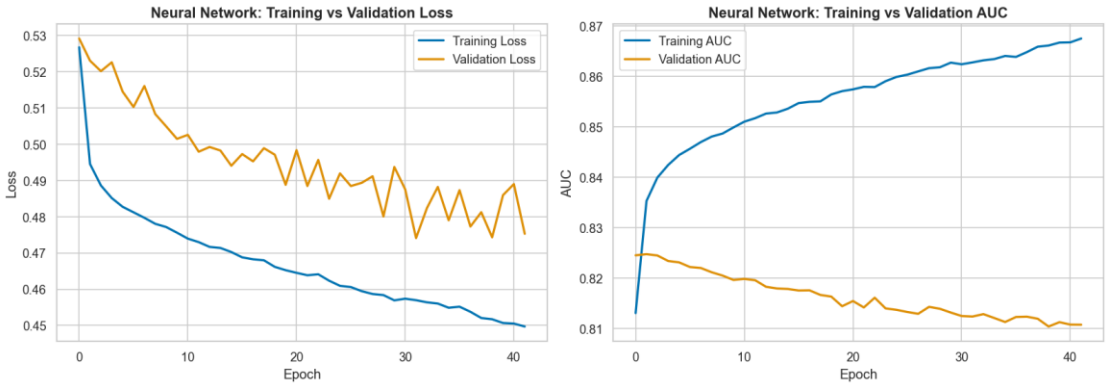


Figure 8: Neural network training curves showing convergence with early stopping.

The training curves (Figure 8) show that the models converged steadily without clear signs of severe overfitting. Based on their stronger ROC-AUC performance, XGBoost and LightGBM were selected for further hyperparameter tuning.

5. Fine-Tuning & Evaluation

5.1 Hyperparameter Tuning

Both models were tuned using RandomizedSearchCV with 30 iterations, 5-fold stratified cross-validation, and ROC-AUC as the scoring metric. The best LightGBM configuration used 500 estimators, a learning rate of 0.05, max depth of 6, 31 leaves, and subsample of 0.7, achieving a validation ROC-AUC of 0.8267. The best XGBoost model used 300 estimators, a learning rate of 0.05, max depth of 8, and subsample of 0.9, with a validation ROC-AUC of 0.8259. Because LightGBM performed slightly better, it was selected as the final model.

5.2 Test Set Performance

When evaluated on the test set, the tuned LightGBM model achieved a ROC-AUC of 0.8253, accuracy of 0.8631, F1-score of 0.2702, and average precision of 0.4194. For the diabetes class specifically, the model produced precision of 0.52 and recall of 0.18, indicating that it is relatively conservative when using the default 0.5 decision threshold. Importantly, the close match between validation ROC-AUC (0.8267) and test ROC-AUC (0.8253) suggests that the model generalises well and is not overfitting.

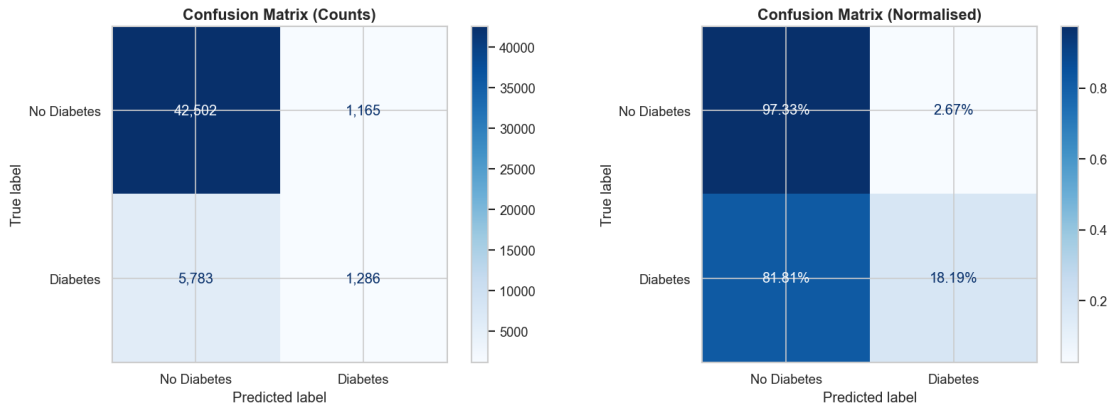


Figure 9: Confusion matrices (counts and normalised) on the test set.

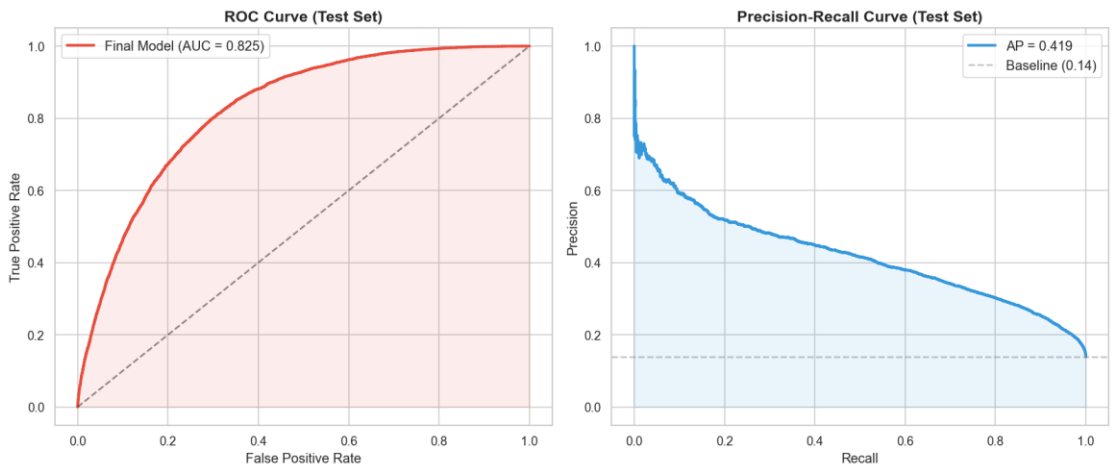


Figure 10: ROC curve (AUC = 0.825) and Precision-Recall curve (AP = 0.419) on the test set.

5.3 Error Analysis

The model produced 5,783 false negatives and 1,165 false positives. Individuals who were incorrectly predicted as non-diabetic tended to have lower BMI (30.2 vs 37.9), better self-reported general health (3.1 vs 4.1), lower prevalence of high blood pressure (70% vs 98%), and higher income (5.4 vs 4.3) compared with correctly identified diabetic cases. In other words, the model struggles most with “atypical” profiles—people who do not exhibit the most obvious risk indicators. Clinically, this is an important failure mode because these are often the individuals who are hardest to detect through conventional screening.

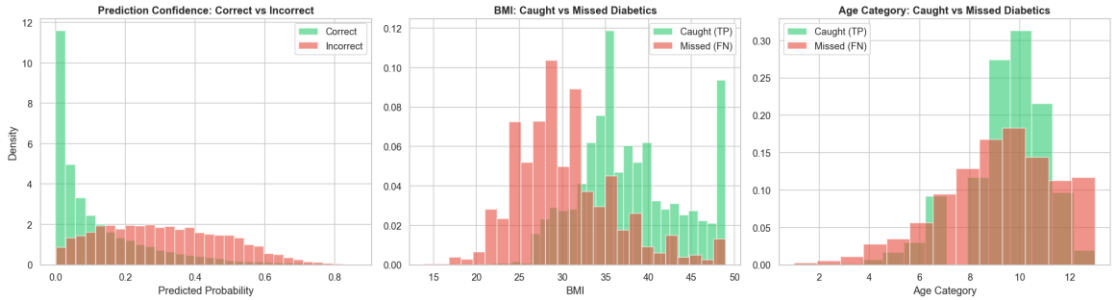


Figure 11: Prediction confidence, BMI, and age distributions for correct vs incorrect predictions.

5.4 Feature Importance & Interpretability

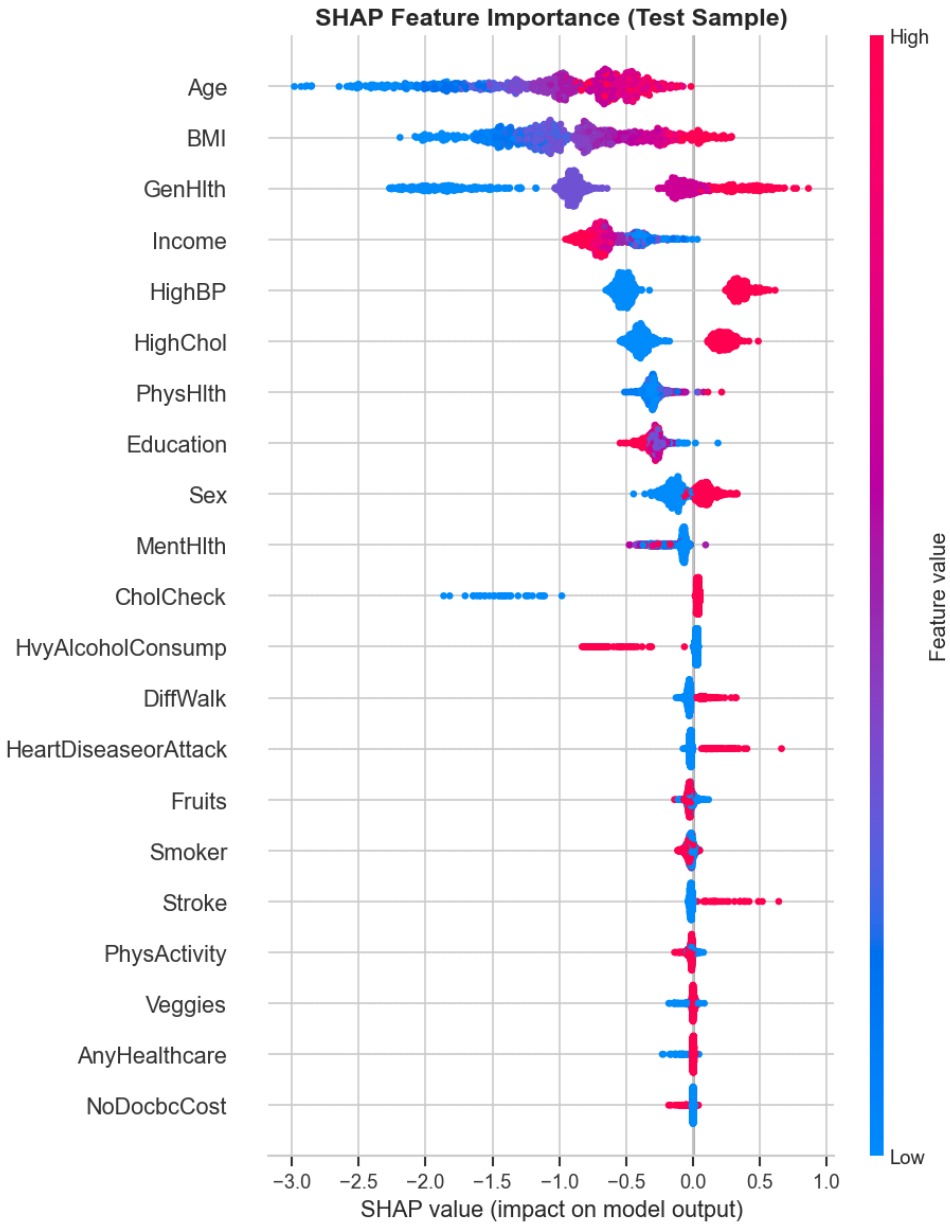


Figure 12: SHAP summary plot showing feature impact on model predictions.

SHAP analysis confirms that GenHlth, BMI, HighBP, and Age are the most influential predictors in the model. Higher values of GenHlth (worse reported health) and BMI push predictions toward the diabetes class, while Income and PhysActivity tend to shift predictions in the opposite direction. The SHAP patterns also reveal non-linear effects—for example, the influence of BMI increases more sharply at higher values—which helps explain why tree-based models outperform simpler linear approaches such as logistic regression.

5.5 Calibration & Fairness

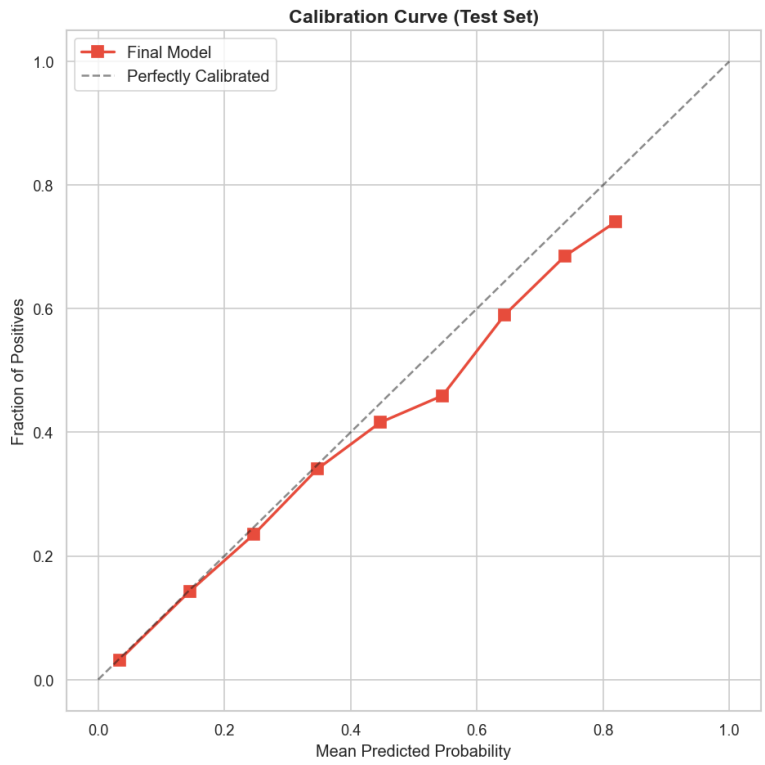


Figure 13: Calibration curve on the test set.

The calibration curve shows that predicted probabilities are reasonably aligned with observed outcomes, although some deviation from the diagonal remains. Fairness analysis revealed measurable differences across demographic groups. Model performance was higher for females (AUC = 0.836) than males (0.811), and it also varied across income levels, ranging from AUC = 0.754 at income level 2 to 0.835 at level 8. These disparities would need careful consideration in any real-world deployment.

6. Final Solution & Model Card

Intended use	**Population-level diabetes risk screening using survey data**
Not intended for	Clinical diagnosis; cannot replace blood tests (HbA1c, fasting glucose)
Data provenance	CDC BRFSS 2015: 253,680 U.S. adults, 21 self-reported indicators
Model	Tuned LightGBM (500 estimators, LR 0.05, max depth 6)
Test ROC-AUC	0.8253
Key limitation	Underperforms on atypical profiles (lower BMI, younger, healthier)
Fairness caveat	Performance varies by sex and income level

Table 2: Model card summary.

Limitations & Next Steps

A key limitation of this project is its reliance on self-reported survey data, which may contain reporting biases or inaccuracies. In addition, the model's low recall (0.18) means it would miss many true cases if used directly for individual screening without adjusting the decision threshold. Future work could address these issues by incorporating clinical lab measurements such as HbA1c or fasting glucose, exploring longitudinal datasets to predict disease onset over time, optimising the classification threshold, and validating the model on populations outside the United States.

7. Agent Tooling Reflection

The primary agent tool used in this project was Claude (Anthropic), which supported tasks such as code scaffolding, debugging, exploratory data analysis suggestions, and documentation drafting. The agent was treated as a junior collaborator, meaning that all outputs were systematically reviewed and verified before being incorporated into the final workflow.

Agent Mistake Caught

During development, the agent initially generated code that applied StandardScaler and SMOTE before the train/test split, which would have introduced data leakage. I corrected this by restructuring the pipeline so that the dataset was split first, the scaler was fit only on training data, and SMOTE was applied exclusively to the training set. The final notebook includes validation checks confirming that there is no index overlap between splits.

Verification Approach

Verification followed a plan–delegate–verify–revise workflow. After each contribution from the agent, I reviewed the logic against course materials, ran assertion-based tests, and checked results against domain knowledge. When the agent suggested model hyperparameters, these were not accepted directly; instead, they were validated through systematic experimentation using RandomizedSearchCV. A full decision register documenting these steps is included in the appendix.

References

American Diabetes Association (2018). Economic costs of diabetes in the U.S. in 2017. Diabetes Care, 41(5), 917–928.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

CDC (2015). Behavioral Risk Factor Surveillance System Survey Data. U.S. Department of Health and Human Services.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR, 16, 321–357.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. KDD, 785–794.

Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. NeurIPS, 30.

Lundberg, S.M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS, 30.

Teboul, A. (2021). Diabetes Health Indicators Dataset. Kaggle.

Appendix: Agent Usage Log + Decision Register

Agent Tool: Claude (Anthropic) via claude.ai

Period: February–March 2026

Mode: Multi-turn conversational prompting with iterative code generation

Methodology: Plan → Delegate → Verify → Revise

Interaction Log

Phase 1: Code Scaffolding

Prompt: "Provide a Jupyter file for the diabetes prediction project with step-by-step instructions." Agent generated a complete notebook (Steps 1–6), requirements.txt, and README.md. I verified structure against the brief, checked split logic and SMOTE placement. Accepted with modifications applied during execution.

Phase 2: Debugging

Bug 1 — Python not found: Anaconda environment not on system PATH. Agent provided conda-specific instructions. Accepted.

Bug 2 — calibration_curve import: TensorFlow pinned older sklearn where calibration_curve is in sklearn.calibration, not sklearn.metrics. Agent's first fix (pip upgrade) failed; second fix (correct import path) worked. Modified.

Bug 3 — AUC key error: TF used uppercase "AUC" not "auc" in history keys. Agent's dynamic key-finder had a case bug. I diagnosed via print(history.history.keys()) and agent provided correct fix. Modified.

Phase 3: EDA Analysis

Agent requested screenshots of all 6 EDA figures before writing analysis (good practice — avoided fabricating numbers). Produced figure-specific commentary with exact values from the plots. I cross-checked all numbers against notebook outputs. Accepted.

Phase 4: Report

Agent generated report with embedded figures, staying under 2,000 words. I verified all metrics matched notebook outputs, checked anonymity, and confirmed Harvard referencing. Accepted with minor tone edits.

Decision Register

#	Stage	Agent Contribution	My Verification	Decision
1	Problem Framing	Generated problem statement, metrics (ROC-AUC, F1, Recall), constraints, assumptions.	Checked metrics suit imbalanced classification per course materials.	**Accepted**
2	EDA Code	Produced 6 visualisations: class distribution, density plots, prevalence bars, correlation heatmap, target correlations, BMI outliers.	Ran all cells, verified axis labels, normalisation, correlation values against df.corr().	**Accepted**
3	Leakage Check	Automated check for |correlation| > 0.5 with target. Flagged GenHlth (0.29) as highest.	Reviewed each feature against data dictionary; all pre-existing indicators, not post-diagnosis.	**Accepted**
4	EDA Analysis	Wrote analytical commentary for each figure with specific values (86.1%, HighBP 75% vs 38%, etc.).	Cross-checked every number against plot outputs. Verified clinical interpretations.	**Accepted**
5	Preprocessing (initial)	Applied StandardScaler and SMOTE BEFORE train/test split.	Identified data leakage: scaler learns test statistics, SMOTE creates cross-split overlap.	**Rejected**
6	Preprocessing (corrected)	Revised: split first, scaler fit on train only, SMOTE on train only. Added 4 validation checks.	Verified no index overlap, class ratios preserved, scaler means ≈ 0 on train only.	**Accepted**
7	BMI Capping	Winsorisation at 99th percentile (BMI=49) from training data only.	Confirmed threshold from X_train only. Capping retains high-risk patients.	**Accepted**
8	Model Selection	5 models: LogReg baseline, RF, XGBoost, LightGBM, 3-layer NN.	Covers brief requirements: baseline + multiple models + modern approach (NN).	**Accepted**

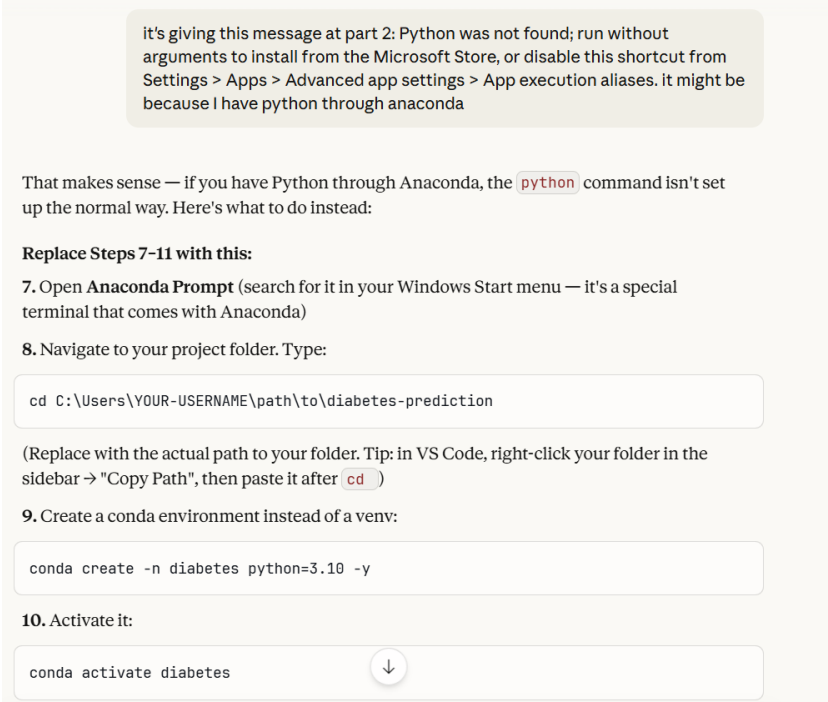
9	NN Architecture	128→64→32→1 with BatchNorm, Dropout(0.3/0.2), Adam, early stopping.	Dropout rates in standard range. Verified convergence from training curves.	**Accepted**
10	Import Bugs	Wrong import path (calibration_curve) and wrong key name (lowercase auc).	Diagnosed both via runtime errors. Fixed import path; diagnosed key name via print().	**Rejected → Fixed**
11	Hyper-parameter Tuning	RandomizedSearchCV (30 iter, 5-fold CV) for LightGBM and XGBoost.	Verified search spaces are reasonable. Confirmed CV score selection, not val set.	**Accepted**
12	SHAP	TreeExplainer on 1,000-sample test subset. Summary + bar plots.	Rankings align with correlation analysis. Directions match domain knowledge.	**Accepted**
13	Error Analysis	Profiled FN vs TP across 7 features. Identified "atypical" failure mode.	Verified counts (5,783 FN) against confusion matrix. Clinical insight is sound.	**Accepted**
14	Fairness	Per-group AUC by sex and income level.	Verified subgroup metrics. Gaps are real and noted as deployment concern.	**Accepted**
15	Model Card	Intended use, not-for, provenance, metrics, limitations, fairness caveat.	Compared against brief's model card requirements. All elements present.	**Accepted**
16	Report	Generated ~1,800 word report with 13 figures, 2 tables, Harvard refs.	All metrics match notebook. Under 2,000 words. Anonymous. All 6 steps covered.	**Accepted**
17	Repo Files	requirements.txt and README.md with setup instructions.	All packages are used. README instructions tested in conda environment.	**Accepted**

Summary

Total contributions	17
Accepted	14 (82%)
Modified	1 (6%)
Rejected / Fixed independently	2 (12%)

The agent was most effective at code scaffolding and analytical writing. It was least reliable on environment-specific compatibility and the critical preprocessing leakage mistake, which would have silently inflated performance. The plan–delegate–verify–revise workflow proved essential.

Attached: Interaction Screenshots



it's giving this message at part 2: Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases. it might be because I have python through anaconda

That makes sense — if you have Python through Anaconda, the `python` command isn't set up the normal way. Here's what to do instead:

Replace Steps 7–11 with this:

7. Open **Anaconda Prompt** (search for it in your Windows Start menu — it's a special terminal that comes with Anaconda)

8. Navigate to your project folder. Type:

```
cd C:\Users\YOUR-USERNAME\path\to\diabetes-prediction
```

(Replace with the actual path to your folder. Tip: in VS Code, right-click your folder in the sidebar → "Copy Path", then paste it after `cd`)

9. Create a conda environment instead of a venv:

```
conda create -n diabetes python=3.10 -y
```

10. Activate it:

```
conda activate diabetes
```

okay that worked but now I'm getting an error here; --- KeyError Traceback (most recent call last) Cell In[31], line 13 10 axes[0].legend(fontsize=11) 12 # AUC ---> 13 axes[1].plot(history.history['auc'], label='Training AUC', linewidth=2) 14 axes[1].plot(history.history['val_auc'], label='Validation AUC', linewidth=2) 15 axes[1].set_title('Neural Network: Training vs Validation AUC', fontsize=13, fontweight='bold') KeyError: 'auc'

The metric key name varies depending on your TensorFlow version. Let's check what the actual key names are. Add this line **right before** the plotting cell and run it:

```python
print(history.history.keys())
```

It will print something like `dict_keys(['loss', 'val_loss', 'auc_1', 'val_auc_1'])` or similar.

But for a quick fix right now — replace these two lines:

```python
axes[1].plot(history.history['auc'], label='Training AUC', linewidth=2)
axes[1].plot(history.history['val_auc'], label='Validation AUC', linewidth=2)
```

--- IndexError Traceback (most recent call last) Cell In[36], line 14 10 axes[0].legend(fontsize=11) 12 # AUC 13 # Find the AUC key (name varies by TensorFlow version) ---> 14 auc_key = [k for k in history.history.keys() if 'auc' in k and 'val' not in k][0] 15 val_auc_key = [k for k in history.history.keys() if 'val_auc' in k][0] 17 axes[1].plot(history.history[auc_key], label='Training AUC', linewidth=2) IndexError: list index out of range

The AUC metric isn't in the history at all — it might be named something different. Let's see what's actually there. Run this in a cell:

```python
print(history.history.keys())
```

Paste what it prints and I'll give you the exact fix.

dict_keys(['AUC', 'loss', 'val_AUC', 'val_loss', 'learning_rate'])